MyOS für Administratoren

Was es ist, wie es arbeitet und wo seine Grenzen liegen · Stand November 2026

MyOS ordnet Dateien nach Projekten, mit überall gleich benannten Standardordnern. Es ist **keine Datenbank, kein Dokumentenmanagementsystem und keine Plattform**. Alles bleibt in gewöhnlichen Dateien und Ordnern auf dem Dateisystem. Die Konfiguration liegt als lesbares Markdown in einem versteckten Ordner `.MyOS/`, ähnlich wie `.git/`. Wer MyOS wieder entfernt, hat einen ordentlich strukturierten Verzeichnisbaum und ein paar Textdateien übrig, sonst nichts.

1. Das Modell in drei Regeln

- **Alles gehört zu einem Projekt.** Ein Projekt ist ein Ordner mit `.MyOS/Project.md`. Projekte enthalten Unterprojekte nach denselben Regeln, beliebig tief.
- **In jedem Projekt dieselben Ordner.** Eine Vorlage legt sie fest, etwa `00_Admin` … `04_Finanz` … `08_Dokumentation`. Die Vorlagen sind gewöhnliche Ordnerbäume und lassen sich je Organisation anpassen.
- **Ein Ordner entsteht erst, wenn er gebraucht wird.** Die Standardordner erscheinen als „blasse Ordner“ (Embryos). Den echten Ordner legt MyOS erst beim ersten Schreiben an, aus der Vorlage und ohne Symlinks. So stehen keine leeren Hüllen im Baum, und die Struktur bleibt trotzdem in jedem Projekt vorhersagbar.

Der Gewinn für den Betrieb: Weil die Abteilungsordner überall gleich heißen, sind Pfade maschinenlesbar. `Projekte/Kunde A/04_Finanz` sagt, wozu eine Datei gehört. Suchen, Skripte, Backups und Berechtigungen können sich an dieser Struktur festhalten.

2. Perspektiven: der gedrehte Baum als Laufwerk

Abgelegt wird nach Projekten, geschaut wird nach Themen. MyOS hängt dafür per FUSE ein Laufwerk `Perspektiven` neben `Projekte` ein. Es zeigt denselben Bestand umgedreht: `Perspektiven/04_Finanz/Projekte/Kunde A` ist dasselbe wie `Projekte/Kunde A/04_Finanz`. Unter `04_Finanz` liegen also die Finanzen aller Projekte nebeneinander, sonst nichts.

- **Jedes Programm kann es.** Der Speichern-Dialog von LibreOffice oder ein Skript schreibt über das Laufwerk, und die Datei landet im echten Projekt. Fehlt der Abteilungsordner dort, entsteht er aus der Vorlage.
- **Nichts wird überdeckt.** Das Laufwerk liegt nicht über `Projekte`. Backups, Git, Dateimanager und Indexer sehen weiter die echten Dateien, und es gibt keine Kopien.
- **Sauberes Aushängen.** Das Laufwerk endet mit der Sitzung. Hält ein Programm noch eine Datei offen, wartet MyOS mit `fusermount -u`, statt lazy auszuhängen, damit nichts in den leeren Mountpoint geschrieben wird. Zugriffe, die die Rechte verbieten, meldet das Laufwerk als verweigert, nicht als „nicht vorhanden“.

3. ACLs: ein Rechtemodell, das Admins schon kennen

Rechte gehören zum Projekt und reisen mit ihm. Sie stehen in `.MyOS/ACLs.md`: Rollen mit Pfaden und Rechten, darunter die Personen mit ihren Rollen. MyOS erfindet dabei kein eigenes Rechtesystem. Das Modell ist **aus Kompatibilitätsgründen** so gebaut, dass es sich eins zu eins auf die nativen ACLs abbilden lässt:

MyOS	Windows (NTFS)	macOS	Linux
Rolle / Person	Gruppe / Benutzer	Gruppe / Benutzer	Gruppe / Benutzer (POSIX-ACLs)
read, write, execute	Lesen, Schreiben, Ausführen	read, write, execute	r, w, x
delete (nicht in write enthalten)	Löschen	delete	w auf dem Elternordner
change (Rechte ändern)	Berechtigungen ändern	writesecurity	nur der Eigentümer

MyOS	Windows (NTFS)	macOS	Linux
Vererbung (live)	Vererbung	beim Anlegen kopiert	Default-ACL, beim Anlegen kopiert
none / weniger Rechte tiefer	Vererbung dort aufheben	ebenso	Eintrag weglassen

```
# Permissions
## Folder
- /{Folder}/: read, write
## Buchhaltung
- /04_Finanz/: read, write, delete
- /07_Arbeit/Personal/: none
# Users
Anna Huber: Buchhaltung
Karl: 07_Arbeit, 04_Finanz
```

Die Regeln im Einzelnen:

- **Vererbung in Unterprojekte.** Ein Unterprojekt ohne eigene ACLs.md steht unter der des nächsten Elternprojekts. Die Regeln unter `## Folder` gelten in jedem Unterprojekt erneut für dessen Standardordner: Wer im Projekt `04_Finanz` bearbeiten darf, darf es auch in `Familie/04_Finanz`. Ein Unterprojekt mit eigener ACLs.md gilt für sich.
- **Die nächste Regel gewinnt, je Rolle.** Eine Regel auf einem tieferen Ordner ersetzt die geerbte, auch wenn sie weniger erlaubt. `none` sperrt eine Rolle aus.
- **Bewusst kein Deny.** Die Rechte einer Person sind die Vereinigung ihrer Rollen. Wer über eine Rolle hinein darf, darf hinein. Das macht das Ergebnis vorhersagbar, ohne Rangfolge von Allow und Deny.
- **Vorlagen tragen Rechte.** Liegt in der Vorlage eine ACLs.md, bekommt jedes neue Projekt sie mit. Die Buchhaltung bekommt `/04_Finanz/` also einmal und dann in jedem Projekt.
- **Robust gepflegt.** Regeln wandern beim Umbenennen und Verschieben mit. Ein Verschieben auf einen Pfad, für den schon Regeln warten, wird abgewiesen. Vor jeder Änderung über die Oberfläche bleibt die vorige ACLs.md datiert liegen, als Verlauf und zum Rückgängigmachen. Geschrieben wird atomar und erst nach einer Schema-Prüfung.

Ehrlich zur Durchsetzung: Heute setzen die Werkzeuge von MyOS die Rechte durch, also der Dateibrowser Scope, die CLI und das Perspektiven-Laufwerk, über eine gemeinsame Policy-Schicht (optional mit Casbin als Backend). Dafür gibt es drei Modi: `off`, `monitor` (prüft und protokolliert, blockiert nichts) und `enforce`. Ein unbekannter Modus fällt auf `enforce` zurück. Am Dateisystem vorbei, etwa in einer Shell, wirken die ACLs noch nicht. Plugins, die die Regeln in NTFS-, APFS- und POSIX-ACLs schreiben, sind vorgesehen, aber noch nicht gebaut; genau dafür ist das Modell kompatibel gehalten. Auf FAT32 und exFAT gibt es keinen Schutz; dafür ist ein Export je Rolle vorgesehen. Und solange es keine Anmeldung gibt, ist „wer bin ich“ eine lokale Zuordnung: Die ACLs verhindern Fehler, aber keinen Vorsatz.

4. Im Vergleich: Rechte in Microsoft Teams

Teams kennt drei Ebenen. Hinter jedem **Team** steht eine Microsoft-365-Gruppe mit den Rollen Besitzer, Mitglied und Gast; alle Standardkanäle liegen als Ordner in einer gemeinsamen SharePoint-Site. **Private und geteilte Kanäle** bekommen je eine eigene Site mit eigener Mitgliederliste, die nur über Teams gepflegt wird. Rechte auf **einzelne Ordner** setzt man in SharePoint, indem man dort die Vererbung unterbricht; Teams zeigt das nicht an, und Microsoft rät selbst davon ab. Begrifflich passt das gut zu MyOS:

Teams / SharePoint	MyOS
Team	Projekt
Besitzer / Mitglied / Gast	Rollen (Besitzer etwa mit change)
privater Kanal: eigene Mitglieder, eigene Site	Unterprojekt mit eigener ACLs.md
Ordner mit unterbrochener Vererbung	tiefere Regel: die nächste gewinnt
Lesen, Bearbeiten (inkl. Löschen), Vollzugriff	read, write, delete (getrennt), change

Der Unterschied liegt in der Struktur. Teams ist flach: keine Unterteams, keine Unterkanäle, und ein privater Kanal übernimmt die Mitglieder des Teams nicht. In MyOS stecken Projekte in Projekten, und die Regeln vererben sich hinein. Vor allem aber gibt es in MyOS **Abteilungsrollen quer über alle Projekte**: „Die Buchhaltung bearbeitet in jedem Projekt die Finanzen, sonst nichts“ ist eine Zeile unter `## Folder` und gilt in jedem Projekt, Unterprojekt und, über die Vorlage, in jedem künftigen. In Teams bleibt dafür, die Buchhalterin in jedes Team aufzunehmen (dann sieht sie alles) oder in jedem Team den Finanzordner in SharePoint abzukoppeln und von Hand zu pflegen.

Was Teams voraus hat: Anmeldung über Entra ID, Durchsetzung auf dem Server und Freigabelinks für einzelne Dateien. Das hat MyOS heute nicht.

5. Was sonst noch dazugehört

- **Papierkorb pro Projekt** in `.MyOS/Trash/`, im freedesktop-Format plus `DeletedBy`. Ein Unterprojekt nimmt seinen Papierkorb mit. Die Haltedauer ist einstellbar (`trash_days`), und einsehen darf ihn nur, wer am Ort löschen durfte.
- **Desk-Profile:** `.MyOS/Desk.md` legt Hintergrund und Farbschema je Projekt fest, über Adapter für KDE, GNOME und Cinnamon. Dabei gewinnt der nächste Vorfahr, und Änderungen des Nutzers bleiben erhalten.
- **Filter und Sortierung** als Markdown (`Filter.md`, `Sort.md`), die nie mehr zeigen, als die ACLs erlauben; **Tags** als Text in erweiterten Attributen (xattr).
- **Export und Import** eines Teilbaums samt `.MyOS/`, Rechten und Vorlagen, als Ordner oder Zip, damit ein Projekt woanders weiterarbeiten kann.
- **CLI für Skripte:** `myproject`, `myls`, `mytag`, `myfilter`, `myexport`, `myimport`.

6. Warum das für den Betrieb interessant ist

- **Kein Lock-in.** Daten und Regeln sind Dateien: grep, Git, diff und rsync funktionieren. Fällt MyOS weg, bleibt alles lesbar.
- **Einheitliche Struktur ohne Zwang.** Jedes Projekt sieht gleich aus, aber nur die Ordner existieren, die gebraucht werden. Rechte hängen an Abteilungsordnern statt an zufällig gewachsenen Pfaden.
- **Selbsterklärend.** Eine ACLs.md lesen auch Menschen, die nie eine Rechteverwaltung geöffnet haben. Das senkt die Zahl der „kannst du mir mal Zugriff geben“-Tickets.
- **Projekte reisen.** Wer einen Projektordner kopiert, auf einen Stick, in eine Cloud oder zu einem Kunden, nimmt Struktur, Regeln, Papierkorb und Personen mit.

7. Technik und Stand

Linux, Python 3 mit PySide6/QML (Dateibrowser Scope, Notizen PostFix), fusepy für Blueprint-Schicht und Laufwerk, optional Casbin. MIT-Lizenz, Quellcode auf GitHub (Palmstroemen/MyOS), rund 1100 automatisierte Tests samt eigener Sicherheitstests.

MyOS ist ein Prototyp und nicht für den Produktivbetrieb freigegeben. Die Kernabläufe laufen stabil, offen sind vor allem Anmeldung, die Abbildung auf native ACLs und die Paketierung für Distributionen. Für einen Pilot empfiehlt sich: Projekte einmal anlegen, ACLs im Modus `monitor` mitlaufen lassen, die Protokolle ansehen und erst dann einzelne Aktionen auf `enforce` stellen.